```
import numpy as np
import pandas as pd
```

Series is 1D and DataFrames are 2D objects

- But why?
- And what exactly is index?

```
# can we have multiple index? Let's try
index_val = [('cse',2019),('cse',2020),('cse',2021),('cse',2022),('ece',2019),('ece',2020),('ece',2021),('ece',2022)]
a = pd.Series([1,2,3,4,5,6,7,8],index=index_val)
a
```

```
(cse, 2019)    1
(cse, 2020)    2
(cse, 2021)    3
(cse, 2022)    4
(ece, 2019)    5
(ece, 2020)    6
(ece, 2021)    7
(ece, 2022)    8
dtype: int64
```

```
# The problem?
a['cse']
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
/usr/local/lib/python3.8/dist-packages/pandas/core/indexes/base.py in get_loc(self, key, method, tolerance)
   3360             try:
-> 3361                 return self._engine.get_loc(casted_key)
   3362             except KeyError as err:

                                 5 frames

pandas/_libs/hashtable_class_helper.pxi in pandas._libs.hashtable.PyObjectHashTable.get_item()

pandas/_libs/hashtable_class_helper.pxi in pandas._libs.hashtable.PyObjectHashTable.get_item()

KeyError: 'cse'

The above exception was the direct cause of the following exception:

KeyError                                  Traceback (most recent call last)
/usr/local/lib/python3.8/dist-packages/pandas/core/indexes/base.py in get_loc(self, key, method, tolerance)
   3361                 return self._engine.get_loc(casted_key)
   3362             except KeyError as err:
-> 3363                 raise KeyError(key) from err
   3364
   3365         if is_scalar(key) and isna(key) and not self.hasnans:

KeyError: 'cse'
```

```
# The solution -> multiindex series(also known as Hierarchical Indexing)
# multiple index levels within a single index
```

```
# how to create multiindex object
# 1. pd.MultiIndex.from_tuples()
index_val = [('cse',2019),('cse',2020),('cse',2021),('cse',2022),('ece',2019),('ece',2020),('ece',2021),('ece',2022)]
multiindex = pd.MultiIndex.from_tuples(index_val)
multiindex.levels[1]
# 2. pd.MultiIndex.from_product()
pd.MultiIndex.from_product([['cse','ece'],[2019,2020,2021,2022]])
```

```
MultiIndex([('cse', 2019),
            ('cse', 2020),
            ('cse', 2021),
            ('cse', 2022),
            ('ece', 2019),
            ('ece', 2020),
            ('ece', 2021),
            ('ece', 2022)],
           )
```

```
# level inside multiindex object


# creating a series with multiindex object
s = pd.Series([1,2,3,4,5,6,7,8],index=multiindex)
s
```

```
cse  2019    1
     2020    2
     2021    3
     2022    4
ece  2019    5
     2020    6
     2021    7
     2022    8
dtype: int64
```

```
# how to fetch items from such a series
s['cse']
```

```
2019    1
2020    2
2021    3
2022    4
dtype: int64
```

```
# a logical question to ask


# unstack
temp = s.unstack()
temp
```

	2019	2020	2021	2022
cse	1	2	3	4
ece	5	6	7	8

```
# stack
temp.stack()
```

```
cse  2019    1
     2020    2
     2021    3
     2022    4
ece  2019    5
     2020    6
     2021    7
     2022    8
dtype: int64
```

```
# Then what was the point of multiindex series?


# multiindex dataframe


branch_df1 = pd.DataFrame(
    [
        [1,2],
        [3,4],
        [5,6],
        [7,8],
        [9,10],
        [11,12],
        [13,14],
        [15,16],
    ],
    index = multiindex,
    columns = ['avg_package','students']
)

branch_df1
```

		avg_package	students
cse	2019	1	2
	2020	3	4
	2021	5	6
	2022	7	8
ece	2019	9	10
	2020	11	12
	2021	13	14
	2022	15	16

```
branch_df1['students']
```

```
cse  2019     2
     2020     4
     2021     6
     2022     8
ece  2019    10
     2020    12
     2021    14
     2022    16
Name: students, dtype: int64
```

```
# Are columns really different from index?
```

```
# multiindex df from columns perspective
branch_df2 = pd.DataFrame(
    [
        [1,2,0,0],
        [3,4,0,0],
        [5,6,0,0],
        [7,8,0,0],
    ],
    index = [2019,2020,2021,2022],
    columns = pd.MultiIndex.from_product([['delhi','mumbai'],['avg_package','students']])
)
```

```
branch_df2
```

	delhi		mumbai	
	avg_package	students	avg_package	students
2019	1	2	0	0
2020	3	4	0	0
2021	5	6	0	0
2022	7	8	0	0

```
branch_df2.loc[2019]
```

```
delhi   avg_package    1
        students       2
mumbai  avg_package    0
        students       0
Name: 2019, dtype: int64
```

```
# Multiindex df in terms of both cols and index
```

```
branch_df3 = pd.DataFrame(
    [
        [1,2,0,0],
        [3,4,0,0],
        [5,6,0,0],
        [7,8,0,0],
        [9,10,0,0],
        [11,12,0,0],
        [13,14,0,0],
        [15,16,0,0],
```

```
    ],
    index = multiindex,
    columns = pd.MultiIndex.from_product([['delhi','mumbai'],['avg_package','students']])
)

branch_df3
```

		delhi		mumbai	
		avg_package	students	avg_package	students
cse	2019	1	2	0	0
	2020	3	4	0	0
	2021	5	6	0	0
	2022	7	8	0	0
ece	2019	9	10	0	0
	2020	11	12	0	0
	2021	13	14	0	0
	2022	15	16	0	0

Stacking and Unstacking

```
branch_df3.stack().stack()
```

```
cse  2019  avg_package  delhi      1
                        mumbai     0
           students     delhi      2
                        mumbai     0
      2020  avg_package  delhi      3
                        mumbai     0
           students     delhi      4
                        mumbai     0
      2021  avg_package  delhi      5
                        mumbai     0
           students     delhi      6
                        mumbai     0
      2022  avg_package  delhi      7
                        mumbai     0
           students     delhi      8
                        mumbai     0
ece  2019  avg_package  delhi      9
                        mumbai     0
           students     delhi     10
                        mumbai     0
      2020  avg_package  delhi     11
                        mumbai     0
           students     delhi     12
                        mumbai     0
      2021  avg_package  delhi     13
                        mumbai     0
           students     delhi     14
                        mumbai     0
      2022  avg_package  delhi     15
                        mumbai     0
           students     delhi     16
                        mumbai     0
dtype: int64
```

Working with multiindex dataframes

```
# head and tail
branch_df3.head()
# shape
branch_df3.shape
# info
branch_df3.info()
# duplicated -> isnull
branch_df3.duplicated()
branch_df3.isnull()
```

```
<class 'pandas.core.frame.DataFrame'>
MultiIndex: 8 entries, ('cse', 2019) to ('ece', 2022)
Data columns (total 4 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   (delhi, avg_package)   8 non-null      int64
 1   (delhi, students)      8 non-null      int64
 2   (mumbai, avg_package)  8 non-null      int64
 3   (mumbai, students)     8 non-null      int64
dtypes: int64(4)
memory usage: 932.0+ bytes
```

		delhi		mumbai	
		avg_package	students	avg_package	students
cse	2019	False	False	False	False
	2020	False	False	False	False
	2021	False	False	False	False
	2022	False	False	False	False
ece	2019	False	False	False	False
	2020	False	False	False	False
	2021	False	False	False	False
	2022	False	False	False	False

```
# Extracting rows single
branch_df3.loc[('cse',2022)]
```

```
delhi   avg_package    7
        students       8
mumbai  avg_package    0
        students       0
Name: (cse, 2022), dtype: int64
```

```
# multiple
branch_df3.loc[('cse',2019):('ece',2020):2]
```

		delhi		mumbai	
		avg_package	students	avg_package	students
cse	2019	1	2	0	0
	2021	5	6	0	0
ece	2019	9	10	0	0

```
# using iloc
branch_df3.iloc[0:5:2]
```

		delhi		mumbai	
		avg_package	students	avg_package	students
cse	2019	1	2	0	0
	2021	5	6	0	0
ece	2019	9	10	0	0

```
# Extracting cols
branch_df3['delhi']['students']
```

```
cse  2019     2
     2020     4
     2021     6
     2022     8
ece  2019    10
     2020    12
     2021    14
     2022    16
Name: students, dtype: int64
```

```
branch_df3.iloc[:,1:3]
```

		delhi	mumbai
		students	avg_package
cse	2019	2	0
	2020	4	0
	2021	6	0
	2022	8	0
ece	2019	10	0
	2020	12	0
	2021	14	0
	2022	16	0

```
# Extracting both
branch_df3.iloc[[0,4],[1,2]]
```

		delhi	mumbai
		students	avg_package
cse	2019	2	0
ece	2019	10	0

```
# sort index
# both -> descending -> diff order
# based on one level
branch_df3.sort_index(ascending=False)
branch_df3.sort_index(ascending=[False,True])
branch_df3.sort_index(level=0,ascending=[False])
```

		delhi		mumbai	
		avg_package	students	avg_package	students
ece	2019	9	10	0	0
	2020	11	12	0	0
	2021	13	14	0	0
	2022	15	16	0	0
cse	2019	1	2	0	0
	2020	3	4	0	0
	2021	5	6	0	0
	2022	7	8	0	0

```
# multiindex dataframe(col) -> transpose
branch_df3.transpose()
```

		cse				ece			
		2019	2020	2021	2022	2019	2020	2021	2022
delhi	avg_package	1	3	5	7	9	11	13	15
	students	2	4	6	8	10	12	14	16
mumbai	avg_package	0	0	0	0	0	0	0	0
	students	0	0	0	0	0	0	0	0

```
# swaplevel
branch_df3.swaplevel(axis=1)
```

		avg_package delhi	students delhi	avg_package mumbai	students mumbai
cse	2019	1	2	0	0
	2020	3	4	0	0
	2021	5	6	0	0
	2022	7	8	0	0
ece	2019	9	10	0	0
	2020	11	12	0	0
	2021	13	14	0	0
	2022	15	16	0	0

Long Vs Wide Data

Name	Height	Weight
John	160	67
Christopher	182	78

Name	Attribute	Value
John	Height	160
John	Weight	67
Christopher	Height	182
Christopher	Weight	78

Wide format is where we have a single row for every data point with multiple columns to hold the values of various attributes.

Long format is where, for each data point we have as many rows as the number of attributes and each row contains the value of a particular attribute for a given data point.

```
# melt -> simple example branch
# wide to long
pd.DataFrame({'cse':[120]}).melt()
```

	variable	value
0	cse	120

```
# melt -> branch with year
pd.DataFrame({'cse':[120],'ece':[100],'mech':[50]}).melt(var_name='branch',value_name='num_students')
```

	branch	num_students
0	cse	120
1	ece	100
2	mech	50

```
pd.DataFrame(
    {
        'branch':['cse','ece','mech'],
        '2020':[100,150,60],
        '2021':[120,130,80],
        '2022':[150,140,70]
    }
).melt(id_vars=['branch'],var_name='year',value_name='students')
```

	branch	year	students
0	cse	2020	100
1	ece	2020	150
2	mech	2020	60
3	cse	2021	120
4	ece	2021	130
5	mech	2021	80
6	cse	2022	150
7	ece	2022	140
8	mech	2022	70

```
# melt -> real world example
death = pd.read_csv('/content/time_series_covid19_deaths_global.csv')
confirm = pd.read_csv('/content/time_series_covid19_confirmed_global.csv')
```

```
death.head()
```

	Province/State	Country/Region	Lat	Long	1/22/20	1/23/20	1/24/20	1/25/20	1/26/20	1/27/20	...	12/24/22	12/25/22	12/
0	NaN	Afghanistan	33.93911	67.709953	0	0	0	0	0	0	...	7845	7846	
1	NaN	Albania	41.15330	20.168300	0	0	0	0	0	0	...	3595	3595	
2	NaN	Algeria	28.03390	1.659600	0	0	0	0	0	0	...	6881	6881	
3	NaN	Andorra	42.50630	1.521800	0	0	0	0	0	0	...	165	165	
4	NaN	Angola	-11.20270	17.873900	0	0	0	0	0	0	...	1928	1928	

5 rows × 1081 columns

```
confirm.head()
```

	Province/State	Country/Region	Lat	Long	1/22/20	1/23/20	1/24/20	1/25/20	1/26/20	1/27/20	...	12/24/22	12/25/22	12/
0	NaN	Afghanistan	33.93911	67.709953	0	0	0	0	0	0	...	207310	207399	2
1	NaN	Albania	41.15330	20.168300	0	0	0	0	0	0	...	333749	333749	3
2	NaN	Algeria	28.03390	1.659600	0	0	0	0	0	0	...	271194	271198	2
3	NaN	Andorra	42.50630	1.521800	0	0	0	0	0	0	...	47686	47686	
4	NaN	Angola	-11.20270	17.873900	0	0	0	0	0	0	...	104973	104973	1

5 rows × 1081 columns

```
death = death.melt(id_vars=['Province/State','Country/Region','Lat','Long'],var_name='date',value_name='num_deaths')
confirm = confirm.melt(id_vars=['Province/State','Country/Region','Lat','Long'],var_name='date',value_name='num_cases')
```

```
death.head()
```

	Province/State	Country/Region	Lat	Long	date	num_deaths
0	NaN	Afghanistan	33.93911	67.709953	1/22/20	0
1	NaN	Albania	41.15330	20.168300	1/22/20	0
2	NaN	Algeria	28.03390	1.659600	1/22/20	0
3	NaN	Andorra	42.50630	1.521800	1/22/20	0
4	NaN	Angola	-11.20270	17.873900	1/22/20	0

```
confirm.merge(death,on=['Province/State','Country/Region','Lat','Long','date'])[['Country/Region','date','num_cases','num_deaths']]
```

	Country/Region	date	num_cases	num_deaths
0	Afghanistan	1/22/20	0	0
1	Albania	1/22/20	0	0
2	Algeria	1/22/20	0	0
3	Andorra	1/22/20	0	0
4	Angola	1/22/20	0	0
...	...	...	...	...
311248	West Bank and Gaza	1/2/23	703228	5708
311249	Winter Olympics 2022	1/2/23	535	0
311250	Yemen	1/2/23	11945	2159
311251	Zambia	1/2/23	334661	4024
311252	Zimbabwe	1/2/23	259981	5637

311253 rows × 4 columns

```
Start coding or generate with AI.
```

```
Start coding or generate with AI.
```

Pivot Table

The pivot table takes simple column-wise data as input, and groups the entries into a two-dimensional table that provides a multidimensional summarization of the data.

```
import numpy as np
import pandas as pd
import seaborn as sns
```

```
df = sns.load_dataset('tips')
df.head()
```

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

```
df.groupby('sex')[['total_bill']].mean()
```

	total_bill
sex	
Male	20.744076
Female	18.056897

```
df.groupby(['sex','smoker'])[['total_bill']].mean().unstack()
```

	total_bill	
smoker	Yes	No
sex		
Male	22.284500	19.791237
Female	17.977879	18.105185

```
df.pivot_table(index='sex',columns='smoker',values='total_bill')
```

smoker	Yes	No
sex		
Male	22.284500	19.791237
Female	17.977879	18.105185

```
Start coding or generate with AI.
```

```
Start coding or generate with AI.
```

```
# aggfunc
df.pivot_table(index='sex',columns='smoker',values='total_bill',aggfunc='std')
```

smoker	Yes	No
sex		
Male	9.911845	8.726566
Female	9.189751	7.286455

```
# all cols together
df.pivot_table(index='sex',columns='smoker')['size']
```

smoker	Yes	No
sex		
Male	2.500000	2.711340
Female	2.242424	2.592593

```
# multidimensional
df.pivot_table(index=['sex','smoker'],columns=['day','time'],aggfunc={'size':'mean','tip':'max','total_bill':'sum'},margins=True)
```

		size							tip			...	total_bill			
	day	Thur		Fri		Sat	Sun	All	Thur		Fri	...	All	Thur		Fri
	time	Lunch	Dinner	Lunch	Dinner	Dinner	Dinner		Lunch	Dinner	Lunch	...		Lunch	Dinner	Lunch
sex	smoker															
Male	Yes	2.300000	NaN	1.666667	2.400000	2.629630	2.600000	2.500000	5.00	NaN	2.20	...	10.0	191.71	0.00	34.16
	No	2.500000	NaN	NaN	2.000000	2.656250	2.883721	2.711340	6.70	NaN	NaN	...	9.0	369.73	0.00	0.00
Female	Yes	2.428571	NaN	2.000000	2.000000	2.200000	2.500000	2.242424	5.00	NaN	3.48	...	6.5	134.53	0.00	39.78
	No	2.500000	2.0	3.000000	2.000000	2.307692	3.071429	2.592593	5.17	3.0	3.00	...	5.2	381.58	18.78	15.98
All		2.459016	2.0	2.000000	2.166667	2.517241	2.842105	2.569672	6.70	3.0	3.48	...	10.0	1077.55	18.78	89.92

5 rows × 23 columns

```
# margins
df.pivot_table(index='sex',columns='smoker',values='total_bill',aggfunc='sum',margins=True)
```

smoker	Yes	No	All
sex			
Male	1337.07	1919.75	3256.82
Female	593.27	977.68	1570.95
All	1930.34	2897.43	4827.77

```
# plotting graphs
df = pd.read_csv('/content/expense_data.csv')
```

```
df.head()
```

	Date	Account	Category	Subcategory	Note	INR	Income/Expense	Note.1	Amount	Currency	Account.1
0	3/2/2022 10:11	CUB - online payment	Food	NaN	Brownie	50.0	Expense	NaN	50.0	INR	50.0
1	3/2/2022 10:11	CUB - online payment	Other	NaN	To lended people	300.0	Expense	NaN	300.0	INR	300.0
2	3/1/2022 19:50	CUB - online payment	Food	NaN	Dinner	78.0	Expense	NaN	78.0	INR	78.0
	3/1/2022	CUB - online									

```
df['Category'].value_counts()
```

```
Food                156
Other                60
Transportation       31
Apparel               7
Household             6
Allowance             6
Social Life           5
Education             1
Salary                1
Self-development      1
Beauty                1
Gift                  1
Petty cash            1
Name: Category, dtype: int64
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 277 entries, 0 to 276
Data columns (total 11 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Date            277 non-null    object
 1   Account         277 non-null    object
 2   Category        277 non-null    object
 3   Subcategory     0 non-null      float64
 4   Note            273 non-null    object
 5   INR             277 non-null    float64
 6   Income/Expense  277 non-null    object
 7   Note.1          0 non-null      float64
 8   Amount          277 non-null    float64
 9   Currency        277 non-null    object
 10  Account.1       277 non-null    float64
dtypes: float64(5), object(6)
memory usage: 23.9+ KB
```

```
df['Date'] = pd.to_datetime(df['Date'])
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 277 entries, 0 to 276
Data columns (total 11 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Date            277 non-null    datetime64[ns]
 1   Account         277 non-null    object
 2   Category        277 non-null    object
 3   Subcategory     0 non-null      float64
```

```
 4   Note            273 non-null    object
 5   INR             277 non-null    float64
 6   Income/Expense  277 non-null    object
 7   Note.1          0 non-null      float64
 8   Amount          277 non-null    float64
 9   Currency        277 non-null    object
 10  Account.1       277 non-null    float64
dtypes: datetime64[ns](1), float64(5), object(5)
memory usage: 23.9+ KB
```

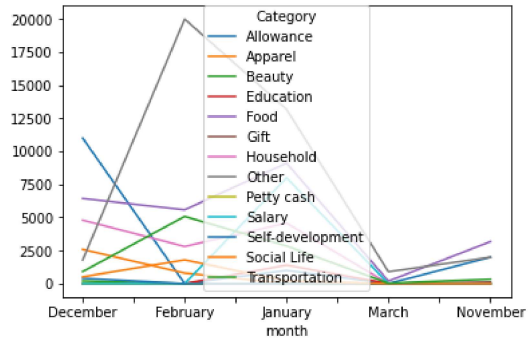
```
df['month'] = df['Date'].dt.month_name()
```

```
df.head()
```

	Date	Account	Category	Subcategory	Note	INR	Income/Expense	Note.1	Amount	Currency	Account.1	month
0	2022-03-02 10:11:00	CUB - online payment	Food	NaN	Brownie	50.0	Expense	NaN	50.0	INR	50.0	March
1	2022-03-02 10:11:00	CUB - online payment	Other	NaN	To lended people	300.0	Expense	NaN	300.0	INR	300.0	March
2	2022-03-01 19:50:00	CUB - online payment	Food	NaN	Dinner	78.0	Expense	NaN	78.0	INR	78.0	March
	2022-03-01	CUB - online										

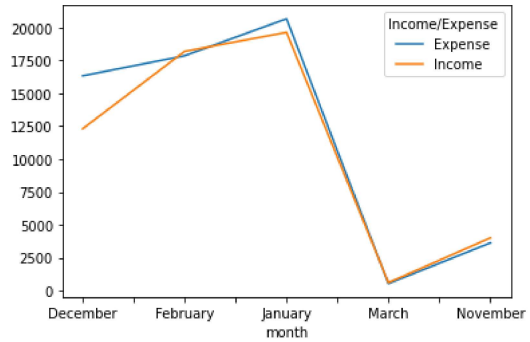
```
df.pivot_table(index='month',columns='Category',values='INR',aggfunc='sum',fill_value=0).plot()
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x7f8976423df0>
```




```
df.pivot_table(index='month',columns='Income/Expense',values='INR',aggfunc='sum',fill_value=0).plot()
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x7f8976118f70>
```




```
df.pivot_table(index='month',columns='Account',values='INR',aggfunc='sum',fill_value=0).plot()
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x7f89760dbe50>
```